

Keyboard Layout Optimization - README

Code Overview: The implementation uses simulated annealing to optimize keyboard layouts by minimizing total finger travel distance. Characters are assigned to fixed QWERTY coordinates, and optimization occurs by swapping character assignments (not positions). Key functions include: `path_length_cost()` which sums Euclidean distances between consecutive characters in text, `create_neighbor()` which generates new layouts by swapping two random characters, `preprocess_text()` which converts text to lowercase and maps unknown characters to space, and `simulated_annealing()` which runs the main optimization loop with geometric temperature cooling ($\text{temp} *= \alpha$ each iteration).

The algorithm follows standard simulated annealing: start with QWERTY assignments, generate neighbors by swapping characters, accept improvements always, accept worse solutions with probability $\exp(-\Delta\text{cost}/\text{temperature})$, cool temperature gradually, and track the best solution found throughout the process.

Figure 1: Cost Optimization Trace

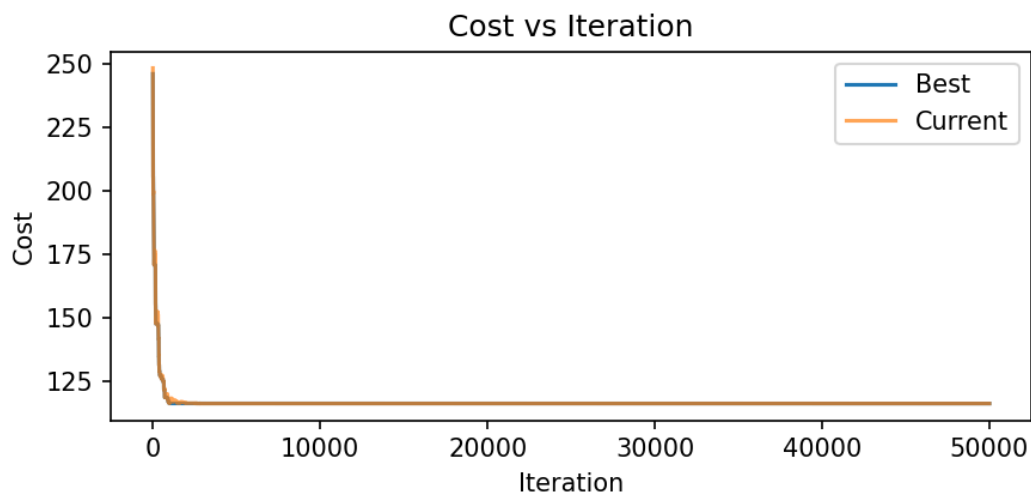
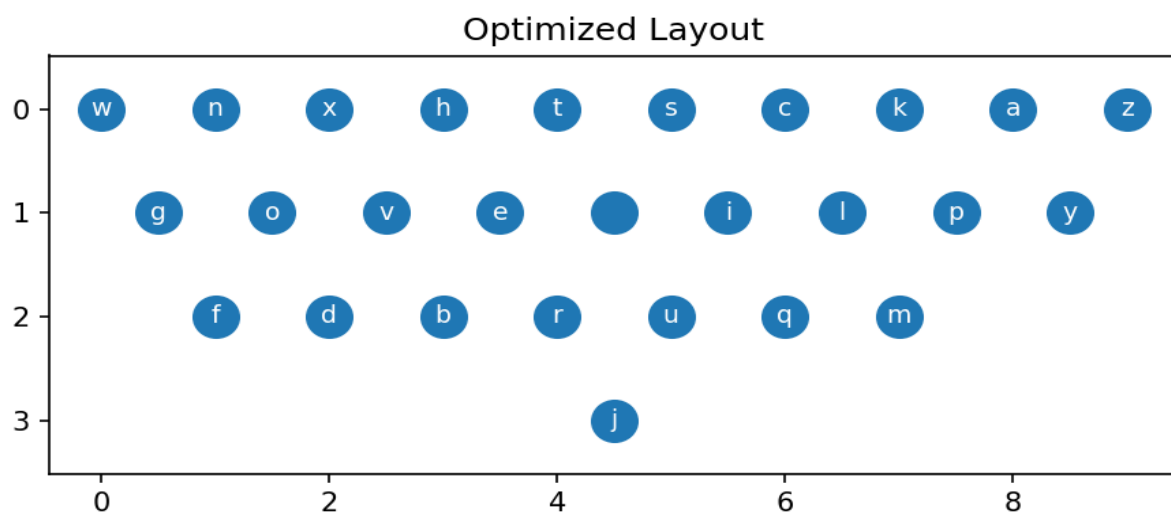


Figure 2: Final Optimized Layout



The cost trace demonstrates the algorithm's behavior: rapid initial improvement as obvious bad swaps are corrected, followed by gradual convergence as fine-tuning occurs. The "best" trace monotonically decreases while "current" trace fluctuates as the algorithm explores worse solutions during cooling phases. The final layout visually shows how frequently co-occurring characters have been clustered closer together, reducing average travel distance compared to the QWERTY baseline.

JSON Format Output: The optimized layout is saved as a dictionary mapping each character to its (x,y) coordinate: `{"a": [0.5, 1.0], "e": [2.0, 0.0], "t": [5.0, 0.0], "h": [5.5, 1.0], " ": [4.5, 3.0], ...}` This allows easy reconstruction of the layout and further analysis.

Parameter Analysis and Effects: Temperature settings significantly impact optimization quality. Higher initial temperature ($t_0=2.0$) allows more exploration of worse solutions early on, potentially finding better global optima but requiring longer convergence time. Lower temperature ($t_0=0.5$) causes faster convergence but increases risk of getting trapped in local optima.

Cooling rate (α) controls exploration vs exploitation tradeoff. Slower cooling ($\alpha=0.9999$) provides better exploration and higher quality solutions but increases runtime significantly. Faster cooling ($\alpha=0.99$) produces quicker results but may miss optimal solutions by cooling too rapidly.

Iteration count affects solution quality for different text lengths. 50,000 iterations provides good balance for typical texts. Longer texts (10,000+ characters) benefit from 100,000+ iterations, while shorter texts may converge earlier with diminishing returns from additional iterations.

Usage Instructions: Basic execution with defaults: `python kbd_optim.py` This runs with built-in text ("the quick brown fox..." + "APL is the best course ever") using 50,000 iterations, $t_0=1.0$, $\alpha=0.999$.

Custom text file: `python kbd_optim.py --file sample_text.txt`
Optimizes layout for provided text file content.

Parameter tuning: `python kbd_optim.py --file mytext.txt --iters 100000 --t0 2.0 --alpha 0.9995` Allows experimentation with different optimization settings.

Available Parameters: `--file` (input text file path), `--iters` (number of iterations, default 50000), `--t0` (initial temperature, default 1.0), `--alpha` (cooling rate per iteration, default 0.999)

Generated Outputs: `layout.json` (final character-to-coordinate mapping), `cost_trace.png` (optimization progress showing both best and current costs), `layout.png` (visual keyboard layout scatter plot), plus console output showing baseline cost, optimized cost, improvement achieved, and total runtime.